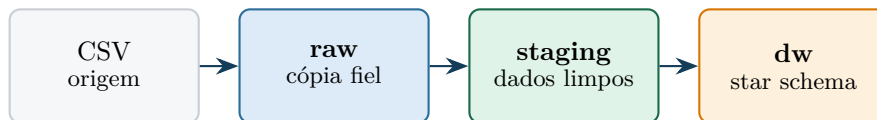


Data Warehouse com PostgreSQL

Construindo um *star schema* passo a passo

a partir do dataset **Supermarket Sales**



PostgreSQL · pgAdmin · SQL · PL/pgSQL

Sumário

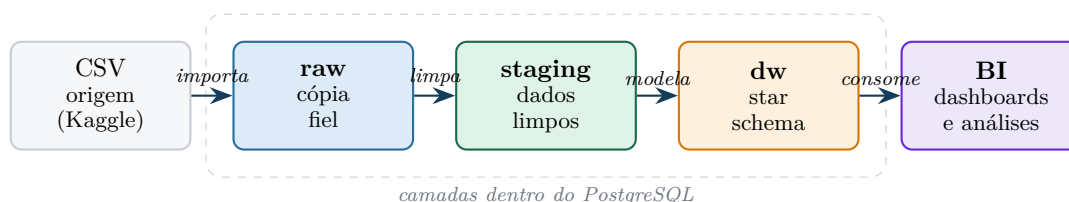
1	O que é um Data Warehouse	1
1.1	Visão geral do fluxo	1
1.2	OLTP vs OLAP	1
1.3	As formas normais: por que o OLTP é “normalizado”	1
1.4	ETL vs ELT	3
2	Modelagem Dimensional (Kimball)	5
2.1	Fatos e Dimensões	5
2.2	Star schema vs Snowflake schema	7
2.3	Granularidade (o “grão”)	9
2.4	Chave surrogate	9
3	Arquitetura em Camadas	10
4	O Dataset e o Ambiente	10
4.1	Supermarket Sales	10
4.2	Ambiente mínimo	10
5	Criando os Schemas	11
6	Camada RAW — criação e importação	11
6.1	Criando a tabela	11
6.2	Importando o CSV pelo pgAdmin	12
7	Camada STAGING — limpeza em SQL	13
7.1	Criando a tabela staging	13
7.2	Construindo o ETL passo a passo	13
8	Modelo Dimensional (camada DW)	16
8.1	Nosso star schema	17
8.2	Entendendo o <code>generate_series</code> antes de usá-lo	17
8.3	Dim date	18
8.4	Dim branch, product, customer e payment	19
8.5	Fact_sales construída em passos	20
9	Automatizando com PL/pgSQL	23
9.1	Um cursor para a curva ABC	23
9.2	Um trigger para auditar a fato	25
10	Consultas Analíticas	27
10.1	P1 — Receita por filial e mês	27
10.2	P2 — Categorias mais lucrativas	28
10.3	P3 — Comportamento por gênero e tipo de cliente	28
10.4	P4 — Dia da semana mais forte	29
11	Cubo OLAP: a última peça	29
11.1	As quatro operações do cubo	30
11.2	Cubo no próprio PostgreSQL: GROUPING SETS, ROLLUP e CUBE	30
11.3	Ferramentas do mercado	31
12	Conclusão	32
12.1	Para onde ir a partir daqui	33

1 O que é um Data Warehouse

Um **Data Warehouse** (DW) é um banco de dados desenhado para *análise*, não para *operação*. Enquanto o sistema do supermercado registra cada venda em milissegundos — um sistema **OLTP** (*Online Transaction Processing*, processamento de transações *online*) —, o DW responde perguntas como “qual filial teve maior lucro no último trimestre?” — um sistema **OLAP** (*Online Analytical Processing*, processamento analítico *online*) — varrendo milhões de linhas.

1.1 Visão geral do fluxo

Antes de mergulhar nos detalhes, veja o caminho completo que os dados percorrem, desde o CSV original até o dashboard de BI (*Business Intelligence*, inteligência de negócio). Cada etapa abaixo é desenvolvida em uma seção própria:



Nota

Esse pipeline — mover dados de uma fonte, passar por estágios de limpeza e modelagem até um consumo final — é o que chamamos de **ETL** ou **ELT**. A diferença entre os dois termos aparece na seção 1.4.

1.2 OLTP vs OLAP

OLTP (operacional)

- Registra transações em tempo real
- Muitos INSERT / UPDATE
- Normalizado (3FN)
- Resposta em milissegundos
- Guarda o estado atual

Ex.: sistema PDV do supermercado

OLAP (analítico)

- Responde perguntas de negócio
- Muitos SELECT pesados
- Desnormalizado (estrela)
- Resposta em segundos
- Guarda histórico completo

Ex.: Data Warehouse

Nota

PDV significa **Ponto de Venda** — o caixa do supermercado, com leitor de código de barras, gaveta de dinheiro e impressora de cupom. Em inglês aparece como **POS** (*Point of Sale*). É o sistema OLTP mais típico do varejo: milhares de pequenas transações por dia, cada uma precisando ser gravada em frações de segundo.

Conceito-chave

A pergunta típica de OLTP é “quanto está este item no estoque agora?”. A de OLAP é “como esse item vendeu mês a mês nos últimos 2 anos, por filial e forma de pagamento?”. Mesmo dado, perguntas diferentes, bancos diferentes.

1.3 As formas normais: por que o OLTP é “normalizado”

Dissemos acima que um banco OLTP é *normalizado* (3FN). Normalizar é organizar as tabelas de modo que cada fato do mundo real seja armazenado **uma única vez**. Isso evita anomalias: se o

endereço de um cliente aparece em 500 linhas de pedidos e ele se muda, ou você atualiza as 500 ou o banco fica inconsistente.

Vamos normalizar um exemplo minúsculo, passo a passo.

Ponto de partida: tabela não normalizada

pedido	cliente	cidade	produtos
101	Ana	São Paulo	Café (2), Pão (3)
102	Bruno	Campinas	Leite (1)
103	Ana	São Paulo	Pão (5), Leite (2), Café (1)

Problema: a coluna **produtos** guarda *vários* valores em uma célula. Não dá para somar quantidades, filtrar por produto ou fazer JOIN.

1FN — Primeira Forma Normal

Regra: todo campo contém um valor atômico (indivisível) e não há grupos repetidos. Cada produto vira uma linha própria.

pedido	item	cliente	cidade	produto	qtd
101	1	Ana	São Paulo	Café	2
101	2	Ana	São Paulo	Pão	3
102	1	Bruno	Campinas	Leite	1
103	1	Ana	São Paulo	Pão	5
103	2	Ana	São Paulo	Leite	2
103	3	Ana	São Paulo	Café	1

A chave primária agora é composta: (**pedido**, **item**). Já dá para consultar, mas “Ana / São Paulo” se repete cinco vezes.

2FN — Segunda Forma Normal

Regra: estar em 1FN e nenhum atributo depender apenas de *parte* da chave composta. Aqui **cliente** e **cidade** dependem só de **pedido**, não do par (**pedido**, **item**) — é uma *dependência parcial*. Separamos em duas tabelas:

pedidos		
pedido	cliente	cidade
101	Ana	São Paulo
102	Bruno	Campinas
103	Ana	São Paulo

itens_pedido			
pedido	item	produto	qtd
101	1	Café	2
101	2	Pão	3
102	1	Leite	1
103	1	Pão	5
103	2	Leite	2
103	3	Café	1

3FN — Terceira Forma Normal

Regra: estar em 2FN e nenhum atributo não-chave depender de outro atributo não-chave (*dependência transitiva*). Suponha que a tabela **pedidos** também guardasse a UF e a região:

pedido	cliente	cidade	uf	região
101	Ana	São Paulo	SP	Sudeste
102	Bruno	Campinas	SP	Sudeste
103	Ana	São Paulo	SP	Sudeste

A UF não depende do pedido: depende da *cidade*. E a região depende da UF. Isso é transitivo. A 3FN manda extrair essas dependências:

pedidos		
pedido	cliente_id	cidade_id
101	1	10
102	2	11
103	1	10

clientes	
cliente_id	nome
1	Ana
2	Bruno

cidades		
cidade_id	nome	uf
10	São Paulo	SP
11	Campinas	SP

ufs	
uf	regiao
SP	Sudeste
RJ	Sudeste

Conceito-chave

Resumo prático das três regras:

- **1FN** — nada de listas dentro de uma célula; cada valor é atômico.
- **2FN** — 1FN + nenhum atributo depende só de parte da chave composta.
- **3FN** — 2FN + nenhum atributo não-chave depende de outro atributo não-chave.

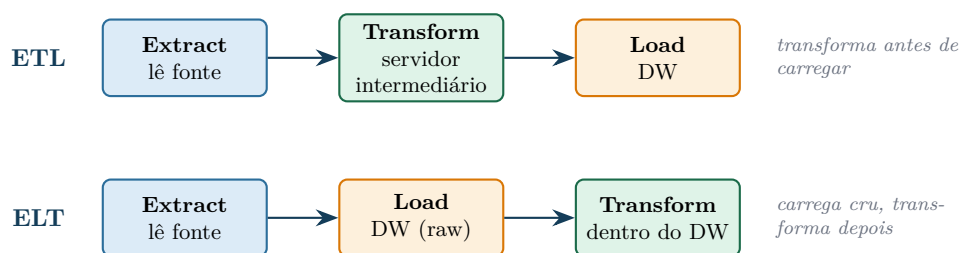
Um mnemônico clássico: *cada atributo depende da chave, da chave inteira, e de nada além da chave.*

No mercado

No OLTP a normalização é obrigatória: ela protege a integridade quando milhares de UPDATE acontecem por segundo. No DW fazemos o **caminho inverso** — *desnormalizamos* de propósito. Ninguém atualiza uma dimensão mil vezes por segundo, e a redundância que seria perigosa no OLTP vira *velocidade* no OLAP, porque elimina JOINS. Entender as formas normais é justamente o que permite decidir com consciência quando abandoná-las.

1.4 ETL vs ELT

Ao mover dados de uma fonte para o DW, sempre passamos por três verbos: **Extract** (extrair), **Transform** (transformar) e **Load** (carregar). A diferença entre ETL e ELT está na **ordem** desses verbos — e essa ordem tem consequências grandes na arquitetura.



Os diagramas acima mostram a *ordem* dos verbos. Mas a diferença prática é sobretudo **onde a máquina trabalha**. As duas figuras seguintes mostram exatamente isso: em que computador cada etapa consome CPU e memória.



Figura 1: ETL — o processamento pesado acontece em uma **máquina intermediária**. O DW recebe apenas o resultado final.

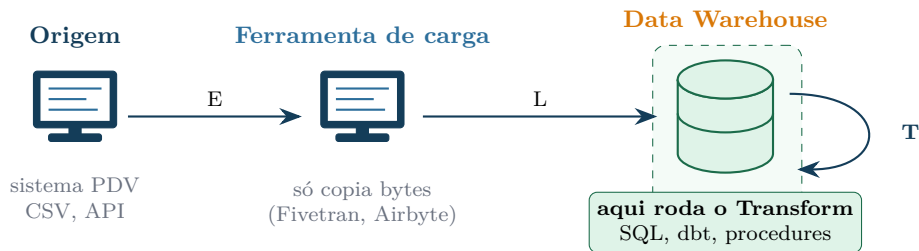


Figura 2: ELT — a máquina do meio só transporta. Todo o processamento pesado acontece **dentro do próprio DW**, usando o poder de cálculo dele.

ETL (*Extract-Transform-Load*) é a abordagem clássica, das décadas de 1990 e 2000:

- Os dados são extraídos da origem, passam por um servidor intermediário (Informatica, Talend, SSIS ou scripts Python) onde são transformados, e só depois entram no DW já limpos.
- Faz sentido quando o DW é caro (*appliances* Teradata, Oracle Exadata) e processar dentro dele custa mais do que num servidor de ETL.
- O DW só recebe o produto final.

ELT (*Extract-Load-Transform*) é a abordagem moderna, dominante desde 2015:

- Os dados são extraídos e carregados *crus* no DW (na camada *raw*). Só depois, já dentro do DW, são transformados via SQL até *staging* e *dw*.
- Faz sentido quando o DW tem processamento barato e elástico (Snowflake, BigQuery, Redshift, PostgreSQL bem dimensionado).
- Ferramentas como o dbt (*data build tool*) popularizaram esse padrão.

	ETL (clássico)	ELT (moderno)
Onde transforma	Servidor intermediário	Dentro do próprio DW
Linguagem	Scripts (Python, ferramenta gráfica)	SQL
DW armazena	Só dado final limpo	Raw + staging + dw
Reprocessamento	Reexecutar pipeline externo	Rodar SQL novamente
Custo típico	Licença ETL + infra dedicada	Só o custo do DW
Escala	Limitada ao servidor de ETL	Elástica (paralelismo do DW)

Conceito-chave

O que faremos aqui é **ETL**: os dados do CSV são *extraídos* do arquivo, *transformados* a cada etapa do pipeline (via SQL, na progressão *raw* → *staging* → *dw*) e o produto final é *carregado* nas tabelas dimensionais prontas para o BI. A camada *raw* é parte do E: ela existe para isolar a leitura do CSV. Tudo o que vem depois dela é transformação ativa, até o *Load* final nas tabelas *dw.**.

A partir da seção 5, um selo no início de cada etapa indica em qual dos três verbos estamos.

2 Modelagem Dimensional (Kimball)

2.1 Fatos e Dimensões

A modelagem dimensional, criada por Ralph Kimball, organiza os dados em dois tipos de tabela:

- **Tabela Fato:** guarda as *medições* (valor, quantidade, nota). Muitas linhas, poucas colunas, predominantemente numérica. Cada linha é um *evento de negócio* — uma venda, por exemplo.
- **Tabela Dimensão:** guarda o *contexto* (quem, o quê, onde, quando). Poucas linhas, muitas colunas, textual. Cada linha descreve um item (um produto, uma data, uma filial).

Ligando uma fato central a várias dimensões, obtemos o **star schema** (esquema em estrela). Para enxergar a estrutura com clareza, vamos desenhá-la como um **diagrama entidade–relacionamento** na notação de **Peter Chen**, a mesma usada em modelagem conceitual clássica:



Figura 3: Os quatro símbolos da notação de Chen usados nas próximas figuras.

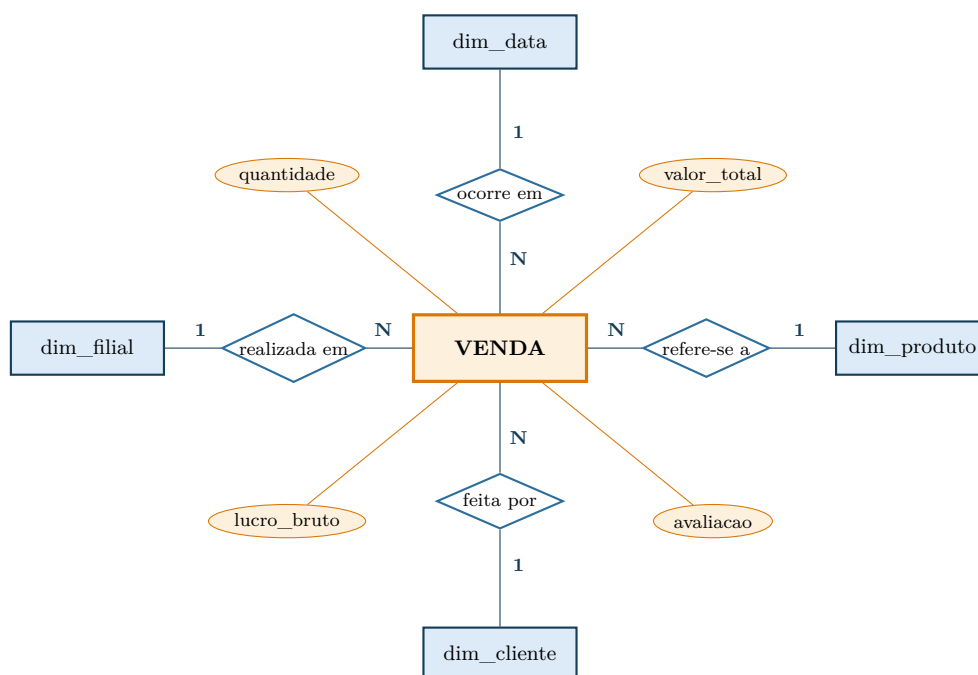


Figura 4: O star schema em notação de Chen. Fato e dimensões são **entidades** (retângulos), sempre ligadas por um **relacionamento** (losango) no meio do caminho. As *métricas* são atributos da entidade **VENDA**. As cardinalidades dizem o essencial: uma venda refere-se a *um* produto (1), e um produto aparece em *muitas* vendas (*N*).

Nota

Leia sempre o diagrama nos dois sentidos, um relacionamento por vez: “*uma venda ocorre em uma data*” e “*uma data tem muitas vendas*”. É esse par de leituras que a cardinalidade 1:N registra — e é ele que, no nível físico, decide de que lado fica a chave estrangeira: sempre

no lado *N*, ou seja, dentro da tabela fato.

Um parêntese: o fato também pode ser desenhado como relacionamento

Vale conhecer a leitura alternativa, porque ela é a mais fiel ao que Chen propôs. Uma tabela fato não tem existência própria: ela só existe para *conectar* dimensões, e suas métricas são justamente o que Chen chamava de **atributos do relacionamento** — o exemplo canônico dele é a **quantidade** escrita no losango entre Fornecedor e Peça. Nessa leitura, VENDA não seria um retângulo, e sim um **relacionamento n-ário** ligando as quatro dimensões de uma vez:

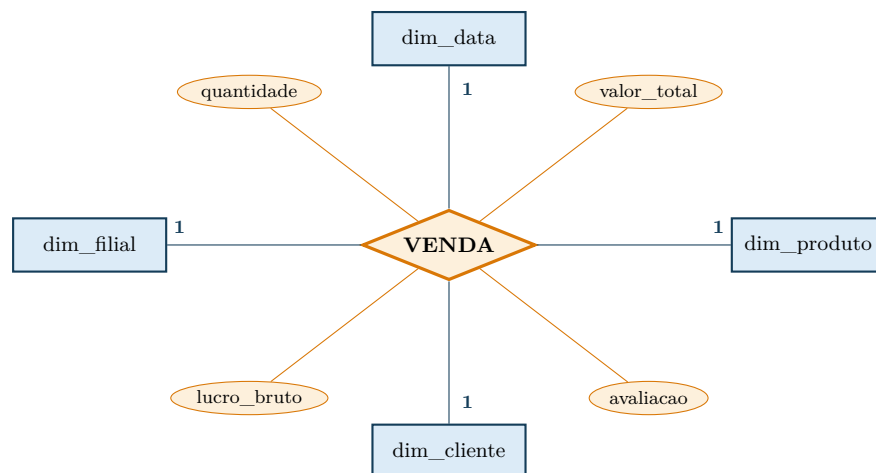


Figura 5: O mesmo star schema visto como relacionamento n-ário: um único losango de grau 4, com as métricas penduradas nele. Aqui o número junto a cada entidade tem outro sentido — indica quantas instâncias daquela dimensão participam de *cada ocorrência* do relacionamento (uma venda envolve exatamente uma data, um produto, um cliente e uma filial).

Os dois diagramas descrevem exatamente o mesmo modelo; a diferença é o que cada um enfatiza:

- O **losango n-ário** é mais fiel à *natureza* do fato — ele deixa explícito que a venda é uma conexão entre dimensões, não uma coisa que existe sozinha.
- A **entidade** (o desenho que usaremos daqui em diante) é mais fiel ao que vai existir no banco: um `CREATE TABLE fact_sales` com cinco chaves estrangeiras. Transformar um relacionamento em entidade tem nome próprio em modelagem: **reificação**, e é o caminho obrigatório quando o relacionamento precisa virar tabela.

Nota

Há ainda um motivo prático para preferir o segundo desenho em sala: quase todo curso apresenta Chen com relacionamentos *binários*, e o olho treinado procura duas caixas nas pontas de cada losango. Um losango de grau 4 exige discutir relacionamentos n-ários antes de discutir star schema — inverte a ordem do assunto. Vale a pena guardar a figura acima para a discussão, e seguir modelando com o fato como entidade.

Conceito-chave

Regra prática: se uma coluna costuma aparecer dentro de `SUM()` ou `AVG()`, é uma *métrica* e mora no fato. Se costuma aparecer em `WHERE` ou `GROUP BY`, é um *atributo* e mora na dimensão.

2.2 Star schema vs Snowflake schema

Quando organizamos fato + dimensões, temos dois desenhos possíveis:

- **Star schema** (estrela): dimensões *desnormalizadas*. Cada dimensão é uma única tabela com todos os seus atributos, mesmo que haja redundância.
- **Snowflake schema** (flocos de neve): dimensões *normalizadas*. Atributos hierárquicos (categoria → subcategoria, cidade → estado → região) são quebrados em tabelas próprias.

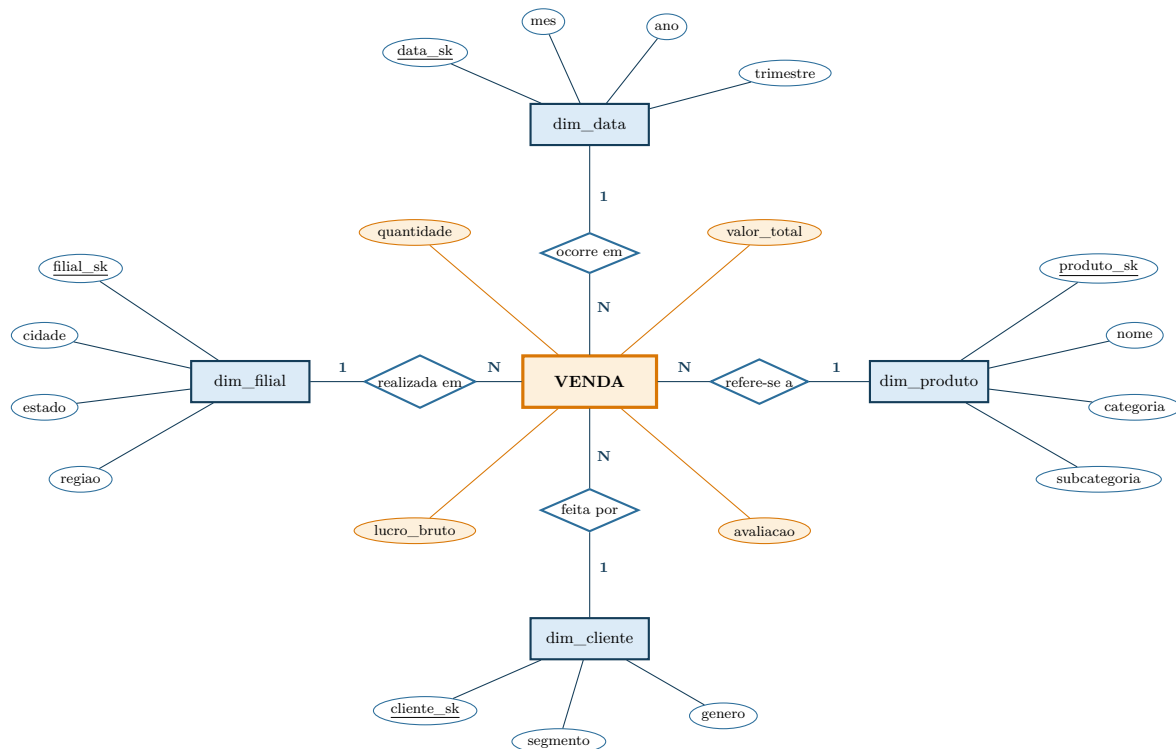


Figura 6: Star schema em notação de Chen: toda a hierarquia vira *atributo* da própria dimensão. categoria e subcategoria penduram-se direto em dim_produto; cidade, estado e regioao, direto em dim_filial. Nenhuma entidade extra.

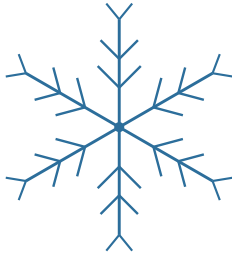
Com dados dentro, o star schema fica assim — repare que dim_produto carrega a categoria *repetida* em várias linhas, e dim_filial repete estado e região:

fato_vendas					
data_sk	produto_sk	filial_sk	qtd	valor	lucro
20190301	1	1	2	48,00	12,00
20190301	3	2	1	15,50	4,10
20190302	1	2	5	120,00	30,00
20190302	2	1	3	27,00	6,30

dim_produto (desnormalizada)			
sk	produto	categoria	subcat.
1	Café	Mercearia	Bebidas
2	Chá	Mercearia	Bebidas
3	Sabão	Limpeza	Roupas

dim_filial (desnormalizada)				
sk	filial	cidade	uf	região
1	Centro	São Paulo	SP	Sudeste
2	Norte	Campinas	SP	Sudeste

Figura 7: Star schema com dados. Um JOIN por dimensão resolve qualquer pergunta: “lucro por região” sai de fato ⋈ dim_filial, direto.



O nome *floco de neve* vem literalmente do desenho ao lado: um núcleo central do qual saem braços que se **ramificam** em braços menores, e esses em outros ainda menores. É exatamente o que acontece com as dimensões normalizadas — a fato no centro, as dimensões em volta e, penduradas nelas, as sub-dimensões de cada nível hierárquico. No *star* os braços são retos e terminam na primeira parada; no *snowflake* eles continuam se abrindo.

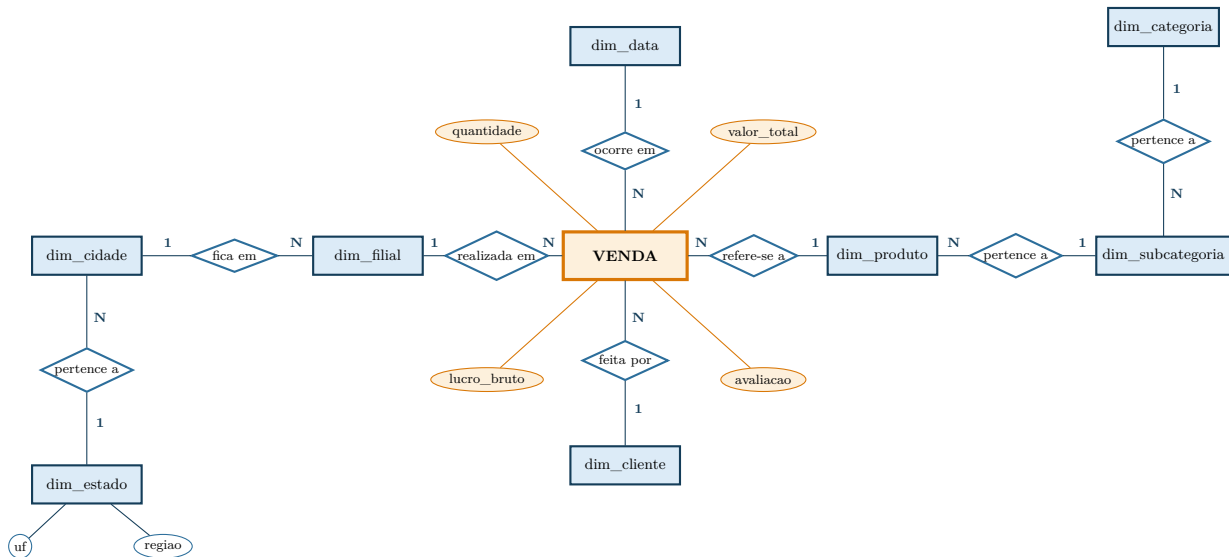


Figura 8: Snowflake schema em notação de Chen: cada nível da hierarquia vira uma *entidade* nova, ligada por um relacionamento próprio. `dim_produto` não guarda mais a categoria — guarda apenas a ligação com `dim_subcategoria`, que por sua vez se liga a `dim_categoria`. É a 3FN aplicada às dimensões.

Os *mesmos dados* do exemplo anterior, agora normalizados. Nenhum texto se repete — mas para chegar da venda até a região são necessários três JOINS encadeados:

dim_produto		
sk	produto	subcat_id
1	Café	100
2	Chá	100
3	Sabão	200

dim_subcategoria		
id	nome	cat_id
100	Bebidas	10
200	Roupas	20

dim_categoria	
id	nome
10	Mercearia
20	Limpeza

dim_filial		
sk	filial	cidade_id
1	Centro	10
2	Norte	11

dim_cidade		
id	nome	estado_id
10	São Paulo	1
11	Campinas	1

dim_estado		
id	uf	região
1	SP	Sudeste
2	RJ	Sudeste

Figura 9: Snowflake com dados: a hierarquia vira uma cadeia de tabelas. “Lucro por região” agora exige `fato` $\bowtie$ `filial` $\bowtie$ `cidade` $\bowtie$ `estado`.

	Star schema	Snowflake schema
Dimensões	Desnormalizadas (redundantes)	Normalizadas (sem redundância)
JOINS necessários	1 por dimensão	Vários (um por nível)
Espaço em disco	Maior (há repetição)	Menor
Performance de query	Melhor (menos JOINS)	Pior
Facilidade para o analista	Alta (uma tabela por assunto)	Baixa (é um quebra-cabeça)
Integridade de hierarquia	Mantida pelo ETL	Garantida pelo modelo

Conceito-chave

A recomendação clássica de Kimball é usar **star**: o analista não deveria precisar de um diagrama para escrever uma query. Snowflake só se justifica em hierarquias muito grandes, com muitas mudanças, ou quando o espaço em disco é crítico. Em DWs modernos, com discos baratos e compressão, o star schema domina amplamente.

Aqui usamos **star schema puro**: a categoria fica dentro de `dim_product`, a cidade dentro de `dim_branch`, o segmento dentro de `dim_customer`. Sem subdimensões, sem complicação.

2.3 Granularidade (o “grão”)

O **grão** é a resposta para a pergunta: “*o que representa uma linha da tabela fato?*”. Definir o grão é a decisão mais importante da modelagem — ele determina o nível de detalhe possível em *todas* as análises futuras. Uma vez agregado, o detalhe está perdido.

Alguns exemplos de grão em contextos diferentes:

- **Supermercado (o nosso caso):** uma linha = *uma venda registrada no cupom*. Se o grão fosse “um item do cupom”, poderíamos analisar produto a produto dentro da mesma compra.
- **E-commerce:** uma linha = *um item de um pedido*. Permite responder “quais produtos costumam ser comprados juntos?”.
- **Banco:** uma linha = *uma transação em conta corrente* (PIX, TED, saque). Um grão mais grosso seria “saldo diário por conta”.
- **Hospital:** uma linha = *um atendimento* (uma consulta ou internação). Mais fino: “um procedimento realizado dentro de um atendimento”.
- **Telecom:** uma linha = *uma chamada telefônica ou sessão de dados*. Mais grosso: “consumo mensal por linha telefônica”.
- **Streaming:** uma linha = *uma sessão de reprodução de um título por um usuário*. Mais fino ainda: “um evento de play/pause/seek”.
- **Logística:** uma linha = *um evento de rastreamento de uma encomenda* (postado, em trânsito, entregue).
- **Escola:** uma linha = *uma nota de um aluno em uma avaliação de uma disciplina*. Mais grosso: “média final por aluno e disciplina”.
- **Indústria:** uma linha = *uma leitura de sensor de uma máquina em um instante*. Esse é um grão bem fino — gera bilhões de linhas por ano.

Conceito-chave

Escolha sempre o **grão mais fino que o negócio consegue fornecer**. É fácil agregar depois (com SUM, GROUP BY ou uma *materialized view*); é impossível desagregar o que já foi somado.

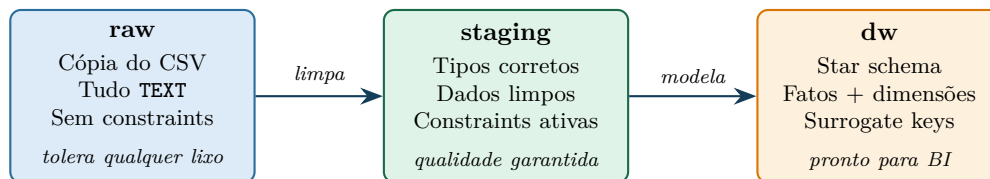
2.4 Chave surrogate

Cada dimensão terá uma chave artificial, sequencial, sem significado de negócio: a **surrogate key** (ex.: `branch_sk` INTEGER). A chave original do sistema de origem (`branch_code` VARCHAR) é preservada como atributo, mas quem o fato referencia é a surrogate. Por quê?

- É menor (inteiro vs. string) → JOINS mais rápidos.
- Isola o DW de mudanças no código da origem (se “A” virar “SP-01”, o DW não quebra).
- Permite implementar histórico (SCD Tipo 2) quando necessário.

3 Arquitetura em Camadas

Nosso DW não será uma tabela só: será organizado em **três schemas** dentro do banco PostgreSQL, cada um com uma responsabilidade clara.



Nota

Por que não importar direto na camada final? Porque dados reais são sujos: strings com espaços, campos vazios, datas em formatos inesperados. Se a tabela final exigir **TIMESTAMP NOT NULL**, a importação falha na primeira linha problemática — e você fica sem *nenhum* dado. A camada **raw** aceita tudo, e o tratamento acontece em SQL, passo a passo, com o dado já seguro dentro do banco.

4 O Dataset e o Ambiente

4.1 Supermarket Sales

Usaremos o dataset **Supermarket Sales** do Kaggle — um único CSV com 1.000 linhas representando vendas fictícias de três filiais de um supermercado em Mianmar (Yangon, Mandalay e Naypyitaw), coletadas entre janeiro e março de 2019.

<https://www.kaggle.com/faresashraf1001/supermarket-sales>

Coluna	Tipo	Descrição
Invoice ID	texto	Identificador único da venda
Branch	texto	Filial (A, B ou C)
City	texto	Cidade da filial
Customer type	texto	Member ou Normal
Gender	texto	Male ou Female
Product line	texto	Categoria (6 valores)
Unit price	número	Preço unitário
Quantity	inteiro	Quantidade
Tax 5%	número	Imposto
Total	número	Valor total pago
Date	data	Data da venda
Time	hora	Hora da venda
Payment	texto	Cash / Credit card / Ewallet
cogs	número	Custo das mercadorias vendidas
gross margin percentage	número	Margem bruta (%)
gross income	número	Lucro bruto
Rating	número	Avaliação do cliente (4 a 10)

4.2 Ambiente mínimo

Só precisamos de duas coisas:

- PostgreSQL 14+ — <https://www.postgresql.org/download/>
- pgAdmin 4+ — <https://www.pgadmin.org/download/>

No pgAdmin, registre o servidor local e crie o banco (*Databases* → *Create* → *Database...*, nome **supermarket_dw**, *Encoding* UTF8).

5 Criando os Schemas

Fase — preparação

Ainda não estamos em E, T ou L: estamos construindo as “prateleiras” onde os dados vão morar. Cada schema corresponde a um estágio do pipeline.

Com o banco `supermarket_dw` criado, abra a *Query Tool* no pgAdmin (ícone de raio com o banco selecionado) e execute:

```
1 CREATE SCHEMA IF NOT EXISTS raw;
2 CREATE SCHEMA IF NOT EXISTS staging;
3 CREATE SCHEMA IF NOT EXISTS dw;
4
5 -- Conferencia
6 SELECT schema_name
7 FROM information_schema.schemata
8 WHERE schema_name IN ('raw','staging','dw');
```

Código 1: Criação dos três schemas

A última consulta deve retornar três linhas. Agora o banco está estruturado para receber os dados.

No mercado

Separar camadas em schemas (ou em *datasets*, no BigQuery, e *databases*, no Snowflake) é padrão de mercado. Times costumam chamá-las de *bronze* / *silver* / *gold* — exatamente as nossas `raw` / `staging` / `dw`. A vantagem prática é o controle de acesso: o analista de negócio recebe permissão só no `dw`, e ninguém consulta por engano uma tabela ainda suja.

6 Camada RAW — criação e importação

Fase E — Extract

Aqui *extraímos* os dados do arquivo CSV e os trazemos para dentro do banco sem alterar absolutamente nada. Nenhuma transformação acontece nesta seção: o objetivo é só garantir que o dado saiu da origem e chegou inteiro.

6.1 Criando a tabela

A tabela `raw.sales` será um espelho fiel do CSV: todas as colunas do tipo `TEXT`, sem nenhuma restrição. É propositalmente permissiva.

```

1 DROP TABLE IF EXISTS raw.sales CASCADE;
2 CREATE TABLE raw.sales (
3     invoice_id          TEXT,
4     branch              TEXT,
5     city                TEXT,
6     customer_type       TEXT,
7     gender              TEXT,
8     product_line        TEXT,
9     unit_price          TEXT,
10    quantity            TEXT,
11    tax_5pct            TEXT,
12    total                TEXT,
13    sale_date            TEXT,
14    sale_time            TEXT,
15    payment              TEXT,
16    cogs                 TEXT,
17    gross_margin_percentage TEXT,
18    gross_income         TEXT,
19    rating               TEXT
20 );

```

Código 2: DDL de raw.sales

Nota

As colunas da tabela seguem a mesma ordem das colunas do CSV. Isso é importante: na importação gráfica do pgAdmin, a ordem é usada para mapear colunas do arquivo para colunas da tabela.

6.2 Importando o CSV pelo pgAdmin

Com a tabela criada, clique com o botão direito em **raw.sales** na árvore de objetos do pgAdmin e selecione *Import/Export Data...*

Aba General

- *Direction*: Import
- *Filename*: selecione o `supermarket_sales.csv`
- *Format*: csv
- *Encoding*: UTF8

Aba Options

- *Header*: ligar
- *Delimiter*: ,
- *Quote*: "
- *Escape*: "

Aba Columns

- Deixe todas marcadas
- Mantenha a ordem
- Clique em *OK*

Após alguns segundos, uma notificação verde indica sucesso. Valide:

```

1 SELECT COUNT(*) FROM raw.sales;      -- deve retornar 1000
2 SELECT * FROM raw.sales LIMIT 3;

```

Código 3: Validação pós-importação

No mercado

Em produção ninguém importa clicando: essa etapa vira um *job* agendado (Airflow, cron, Fivetran) que roda todo dia de madrugada. Mas o princípio é idêntico ao que fizemos aqui — copiar a origem para dentro do banco *sem interpretar nada*. Se a carga falhar, você quer que ela falhe por problema de rede, não por causa de uma vírgula fora do lugar na linha 843.

7 Camada STAGING — limpeza em SQL

Fase T — Transform

Começa a transformação. Os dados já estão no banco, mas todos como texto e sem garantia nenhuma de qualidade. Nesta seção convertemos tipos, padronizamos textos e aplicamos constraints — é aqui que o dado “vira dado”.

7.1 Criando a tabela staging

```
1 DROP TABLE IF EXISTS staging.sales CASCADE;
2 CREATE TABLE staging.sales (
3     invoice_id    VARCHAR(20) PRIMARY KEY,
4     branch        CHAR(1)      NOT NULL,
5     city          VARCHAR(40)   NOT NULL,
6     customer_type VARCHAR(10)   NOT NULL,
7     gender        VARCHAR(10)   NOT NULL,
8     product_line  VARCHAR(40)   NOT NULL,
9     unit_price    NUMERIC(10,2) NOT NULL,
10    quantity      INTEGER       NOT NULL,
11    tax_5pct      NUMERIC(10,4) NOT NULL,
12    total         NUMERIC(12,2) NOT NULL,
13    sale_ts       TIMESTAMP      NOT NULL,
14    payment       VARCHAR(20)    NOT NULL,
15    cogs          NUMERIC(12,2) NOT NULL,
16    gross_income  NUMERIC(10,4) NOT NULL,
17    rating        NUMERIC(4,1)   NOT NULL
18 );
```

Código 4: DDL de staging.sales

Nota

Reduzimos 17 colunas para 15: juntamos `sale_date` e `sale_time` em `sale_ts` (TIMESTAMP) e descartamos `gross_margin_percentage` — é sempre o mesmo valor ($\approx 4,76\%$), não tem uso analítico.

7.2 Construindo o ETL passo a passo

Vamos montar o INSERT em quatro passos, cada um acrescentando um bloco de colunas. As linhas acrescentadas em cada versão levam o comentário `-- novo` à direita.

Passo 1 — apenas identificação

Começamos mapeando o `invoice_id`, já aplicando TRIM:

```
1 INSERT INTO staging.sales (invoice_id)
2 SELECT TRIM(invoice_id)
3 FROM raw.sales;
```

Código 5: ETL staging — Passo 1

Conceito-chave

A função `TRIM(texto)` remove os espaços em branco do **início e do fim** de uma string — sem mexer nos espaços do meio. Ela existe porque exportações de CSV frequentemente deixam espaços invisíveis grudados nos valores:

Entrada	Resultado
TRIM(' Yangon ')	'Yangon'
TRIM('Credit card ')	'Credit card'
TRIM(' 750-67-8428')	'750-67-8428'

Sem TRIM, 'Yangon' e 'Yangon ' seriam *cidades diferentes* para o banco — e a sua dimensão de filiais apareceria com seis linhas em vez de três. Variantes úteis: LTRIM (só à esquerda), RTRIM (só à direita) e TRIM(BOTH 'x' FROM texto) para remover um caractere específico.

Nota

Esse INSERT ainda falharia em produção — todas as outras colunas são NOT NULL. A intenção aqui é didática: cada passo ilustra a transformação de um bloco. Na versão final, tudo vem em um único INSERT. Por enquanto, execute apenas o SELECT para inspecionar o resultado.

Passo 2 — campos categóricos

O que muda neste passo

Surgem os 6 campos categóricos (branch, city, customer_type, gender, product_line, payment) e suas transformações com UPPER e INITCAP.

```

1 INSERT INTO staging.sales (
2     invoice_id,
3     branch, city, customer_type,           -- novo
4     gender, product_line, payment         -- novo
5 )
6 SELECT
7     TRIM(invoice_id),
8     UPPER(TRIM(branch)),                  -- novo
9     INITCAP(TRIM(city)),                  -- novo
10    INITCAP(TRIM(customer_type)),          -- novo
11    INITCAP(TRIM(gender)),                 -- novo
12    INITCAP(TRIM(product_line)),           -- novo
13    INITCAP(TRIM(payment))                 -- novo
14 FROM raw.sales;
```

Código 6: ETL staging — Passo 2 (adiciona categóricos)

Conceito-chave

UPPER / LOWER / INITCAP padronizam a capitalização. INITCAP('CREDIT CARD') devolve 'Credit Card'. Isso garante que credit card e Credit Card sejam tratados como o *mesmo* valor — e é o que impede que a dimensão de pagamento tenha linhas duplicadas.

Passo 3 — valores numéricos com CAST

O que muda neste passo

Entram as 7 colunas numéricas (unit_price, quantity, tax_5pct, total, cogs, gross_income, rating), cada uma convertida com CAST(TRIM(...) AS tipo).


```

1 INSERT INTO staging.sales (
2     invoice_id, branch, city, customer_type,
3     gender, product_line, payment,
4     unit_price, quantity, tax_5pct,          -- novo
5     total, cogs, gross_income, rating        -- novo
6 )
7 SELECT
8     TRIM(invoice_id),
9     UPPER(TRIM(branch)),
10    INITCAP(TRIM(city)),
11    INITCAP(TRIM(customer_type)),
12    INITCAP(TRIM(gender)),
13    INITCAP(TRIM(product_line)),
14    INITCAP(TRIM(payment)),
15    CAST(TRIM(unit_price) AS NUMERIC(10,2)), -- novo
16    CAST(TRIM(quantity) AS INTEGER),         -- novo
17    CAST(TRIM(tax_5pct) AS NUMERIC(10,4)),   -- novo
18    CAST(TRIM(total) AS NUMERIC(12,2)),      -- novo
19    CAST(TRIM(cogs) AS NUMERIC(12,2)),       -- novo
20    CAST(TRIM(gross_income) AS NUMERIC(10,4)), -- novo
21    CAST(TRIM(rating) AS NUMERIC(4,1))       -- novo
22 FROM raw.sales;

```

Código 7: ETL staging — Passo 3 (adiciona numéricos)

Passo 4 — data e hora unidas

O CSV traz `sale_date` (1/5/2019) e `sale_time` (13:08) em colunas separadas. Vamos concatenar e converter para `TIMESTAMP`.

O que muda neste passo

Entra o timestamp completo (`sale_ts`) via `TO_TIMESTAMP`, concatenando data e hora. Também aparecem o `TRUNCATE` no início (para reexecutar sem duplicar) e um `WHERE` defensivo ao final.

```

1 TRUNCATE TABLE staging.sales; -- novo
2
3 INSERT INTO staging.sales (
4     invoice_id, branch, city, customer_type,
5     gender, product_line, payment,
6     unit_price, quantity, tax_5pct,
7     total, cogs, gross_income, rating,
8     sale_ts -- novo
9 )
10 SELECT
11     TRIM(invoice_id),
12     UPPER(TRIM(branch)),
13     INITCAP(TRIM(city)),
14     INITCAP(TRIM(customer_type)),
15     INITCAP(TRIM(gender)),
16     INITCAP(TRIM(product_line)),
17     INITCAP(TRIM(payment)),
18     CAST(TRIM(unit_price) AS NUMERIC(10,2)),
19     CAST(TRIM(quantity) AS INTEGER),
20     CAST(TRIM(tax_5pct) AS NUMERIC(10,4)),
21     CAST(TRIM(total) AS NUMERIC(12,2)),
22     CAST(TRIM(cogs) AS NUMERIC(12,2)),
23     CAST(TRIM(gross_income) AS NUMERIC(10,4)),
24     CAST(TRIM(rating) AS NUMERIC(4,1)),
25     TO_TIMESTAMP( -- novo
26         TRIM(sale_date) || ' ' || TRIM(sale_time), -- novo
27         'MM/DD/YYYY HH24:MI' -- novo
28     ) -- novo
29 FROM raw.sales
30 WHERE TRIM(invoice_id) <> ''; -- novo
31
32 -- Conferencia
33 SELECT COUNT(*) FROM staging.sales; -- 1000
34 SELECT * FROM staging.sales LIMIT 3;

```

Código 8: ETL staging — Passo 4 (versão final)

Conceito-chave

A função `TO_TIMESTAMP(texto, formato)` converte uma string para `TIMESTAMP` conforme o modelo indicado. Códigos úteis: `YYYY` (ano com 4 dígitos), `MM` (mês), `DD` (dia), `HH24` (hora de 0 a 23), `MI` (minutos). O dataset Supermarket Sales usa datas no formato americano `MM/DD/YYYY` — trocar por `DD/MM/YYYY` faria 1º de maio virar 5 de janeiro silenciosamente.

No mercado

Repare que *nenhuma linha foi descartada* na transformação: 1.000 entraram, 1.000 saíram. Em projetos reais isso quase nunca acontece, e times maduros registram a diferença numa tabela de log (quantas linhas rejeitadas, por qual motivo). Essa contagem entrada-vs-saída é a métrica de qualidade mais simples e mais usada de um pipeline.

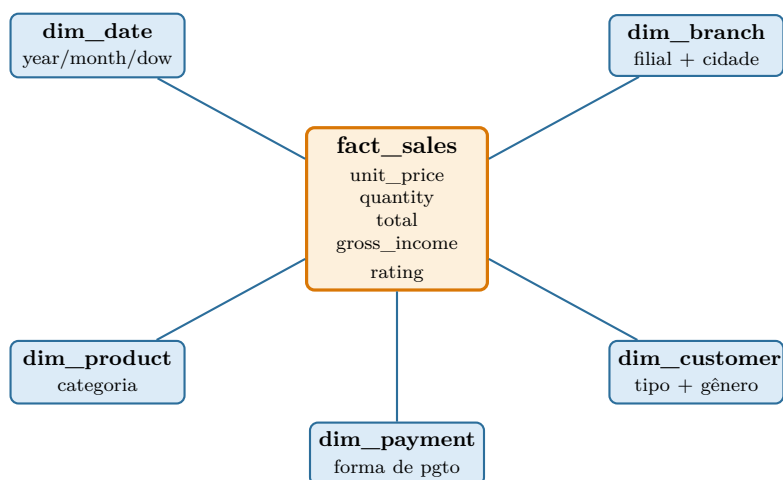
8 Modelo Dimensional (camada DW)

Fase T + L — Transform e Load

Esta é a etapa final do pipeline. Ainda há transformação (separar contexto de métrica, gerar surrogate keys), mas o resultado já é o *Load*: as tabelas que o BI vai consumir. Quando esta seção terminar, o dado estará “entregue”.

8.1 Nosso star schema

Olhando para os dados, identificamos as dimensões (contextos descritivos) e a fato (evento mensurável):



Nota

A `dim_customer` aqui é uma *dimensão de segmento*: o dataset não tem identificador único de cliente, só informa tipo (Member/Normal) e gênero. Isso gera 4 combinações. Em um cenário real com ID de cliente, teríamos milhares de linhas e possivelmente SCD Tipo 2.

8.2 Entendendo o `generate_series` antes de usá-lo

A `dim_date` é especial: ela **não vem da origem**. Nós a *fabricamos*, e a ferramenta para isso é a função `generate_series`, que produz uma sequência de valores — uma linha para cada elemento.

Ela funciona com números:

```
1 SELECT * FROM generate_series(1, 5);
2 SELECT * FROM generate_series(0, 100, 25); -- com passo de 25
```

Código 9: `generate_series` com números

`generate_series(1, 5)`

generate_series
1
2
3
4
5

`generate_series(0, 100, 25)`

generate_series
0
25
50
75
100

Figura 10: Com números, o terceiro argumento é o *passo*. Sem ele, o padrão é 1.

E funciona com datas, onde o passo é um `INTERVAL`:

```
1 SELECT d
2 FROM generate_series(DATE '2019-01-01',
3                     DATE '2019-01-05',
4                     INTERVAL '1 day') g(d);
```

Código 10: `generate_series` com datas

INTERVAL '1 day'	INTERVAL '1 month' (jan-mai)
d	d
2019-01-01 00:00:00	2019-01-01 00:00:00
2019-01-02 00:00:00	2019-02-01 00:00:00
2019-01-03 00:00:00	2019-03-01 00:00:00
2019-01-04 00:00:00	2019-04-01 00:00:00
2019-01-05 00:00:00	2019-05-01 00:00:00

Figura 11: Com datas, o passo é um INTERVAL. Trocando '1 day' por '1 month' ou '1 hour', a mesma função gera calendários mensais ou séries horárias.

O trecho `g(d)` é apenas um *alias*: batiza a tabela resultante de `g` e sua única coluna de `d`, para podermos escrever `d` no SELECT.

No mercado

Gerar a dimensão de tempo em vez de extraí-la é prática universal. O motivo é que o calendário precisa conter *todos* os dias, inclusive aqueles em que não houve nenhuma venda — só assim um relatório mostra “fevereiro: R\$ 0” em vez de simplesmente omitir fevereiro. Em empresas, a `dim_date` costuma ganhar colunas de feriado nacional, semana fiscal, período de campanha e dia útil. É a dimensão mais reaproveitada de todo o DW: um único calendário serve a todas as fatos.

8.3 Dim date

```

1 DROP TABLE IF EXISTS dw.dim_date CASCADE;
2 CREATE TABLE dw.dim_date (
3     date_sk      INTEGER PRIMARY KEY,          -- YYYYMMDD
4     full_date    DATE      NOT NULL UNIQUE,
5     day          SMALLINT  NOT NULL,
6     month        SMALLINT  NOT NULL,
7     month_name   VARCHAR(15) NOT NULL,
8     quarter      SMALLINT  NOT NULL,
9     year         SMALLINT  NOT NULL,
10    day_of_week   VARCHAR(15) NOT NULL,
11    is_weekend    BOOLEAN   NOT NULL
12 );
13
14 INSERT INTO dw.dim_date
15 SELECT
16     CAST(TO_CHAR(d, 'YYYYMMDD') AS INTEGER),
17     d::DATE,
18     EXTRACT(DAY FROM d)::SMALLINT,
19     EXTRACT(MONTH FROM d)::SMALLINT,
20     TO_CHAR(d, 'TMMonth'),
21     EXTRACT(QUARTER FROM d)::SMALLINT,
22     EXTRACT(YEAR FROM d)::SMALLINT,
23     TO_CHAR(d, 'TMDay'),
24     EXTRACT(DOW FROM d) IN (0, 6)
25 FROM generate_series(
26     DATE '2019-01-01', DATE '2019-12-31', INTERVAL '1 day') g(d);
27 SELECT COUNT(*) FROM dw.dim_date; -- 365

```

Código 11: DDL e carga de `dw.dim_date`

Conceito-chave

A chave surrogate da `dim_date`, por convenção, é o inteiro YYYYMMDD (ex.: 20190315). É a única exceção à regra “surrogate não tem significado de negócio” — a convenção é tão útil

(permite ordenar, filtrar faixas e até particionar por ela) que virou padrão de mercado.

8.4 Dim branch, product, customer e payment

São dimensões pequenas, então criamos as quatro juntas:

```

1  -- dim_branch -----
2  DROP TABLE IF EXISTS dw.dim_branch CASCADE;
3  CREATE TABLE dw.dim_branch (
4      branch_sk    SERIAL PRIMARY KEY,
5      branch_code  CHAR(1)      NOT NULL UNIQUE,
6      city         VARCHAR(40) NOT NULL
7  );
8
9  -- dim_product -----
10 DROP TABLE IF EXISTS dw.dim_product CASCADE;
11 CREATE TABLE dw.dim_product (
12     product_sk    SERIAL PRIMARY KEY,
13     product_line  VARCHAR(40) NOT NULL UNIQUE
14 );
15
16 -- dim_customer (segmento) -----
17 DROP TABLE IF EXISTS dw.dim_customer CASCADE;
18 CREATE TABLE dw.dim_customer (
19     customer_sk    SERIAL PRIMARY KEY,
20     customer_type  VARCHAR(10) NOT NULL,
21     gender         VARCHAR(10) NOT NULL,
22     UNIQUE (customer_type, gender)
23 );
24
25 -- dim_payment -----
26 DROP TABLE IF EXISTS dw.dim_payment CASCADE;
27 CREATE TABLE dw.dim_payment (
28     payment_sk     SERIAL PRIMARY KEY,
29     payment_type   VARCHAR(20) NOT NULL UNIQUE
30 );

```

Código 12: DDL das demais dimensões

A carga de cada dimensão é um `DISTINCT` sobre a staging — pegamos os valores únicos de cada contexto:

```

1  INSERT INTO dw.dim_branch (branch_code, city)
2  SELECT DISTINCT branch, city FROM staging.sales;
3
4  INSERT INTO dw.dim_product (product_line)
5  SELECT DISTINCT product_line FROM staging.sales;
6
7  INSERT INTO dw.dim_customer (customer_type, gender)
8  SELECT DISTINCT customer_type, gender FROM staging.sales;
9
10 INSERT INTO dw.dim_payment (payment_type)
11 SELECT DISTINCT payment FROM staging.sales;

```

Código 13: Cargas das dimensões

Para conferir tudo de uma vez, queremos empilhar quatro contagens em uma única saída. Quem faz isso é o `UNION ALL`:

```

1 SELECT 'branch' AS dimensao, COUNT(*) AS linhas FROM dw.dim_branch
2 UNION ALL
3 SELECT 'product', COUNT(*) FROM dw.dim_product
4 UNION ALL
5 SELECT 'customer', COUNT(*) FROM dw.dim_customer
6 UNION ALL
7 SELECT 'payment', COUNT(*) FROM dw.dim_payment;

```

Código 14: Conferência das dimensões com UNION ALL

Conceito-chave

UNION e **UNION ALL** empilham verticalmente o resultado de dois ou mais **SELECTs**. Não confunda com **JOIN**: o **JOIN** cola tabelas *lado a lado* (acrescenta colunas); o **UNION** cola *uma embaixo da outra* (acrescenta linhas). Para funcionar, todos os **SELECTs** precisam ter a mesma quantidade de colunas e tipos compatíveis; os nomes das colunas vêm do primeiro **SELECT**.

A diferença entre os dois:

- **UNION ALL** — devolve **tudo**, inclusive linhas repetidas. É uma simples concatenação.
- **UNION** — devolve o resultado **sem duplicatas**. Para isso o banco precisa ordenar ou construir um hash de todo o conjunto, o que custa tempo e memória.

Consulta A	Consulta B	A UNION ALL B	A UNION B																		
<table><tr><td>v</td></tr><tr><td>1</td></tr><tr><td>2</td></tr><tr><td>3</td></tr></table>	v	1	2	3	<table><tr><td>v</td></tr><tr><td>3</td></tr><tr><td>4</td></tr></table>	v	3	4	<table><tr><td>v</td></tr><tr><td>1</td></tr><tr><td>2</td></tr><tr><td>3</td></tr><tr><td>3</td></tr><tr><td>4</td></tr></table>	v	1	2	3	3	4	<table><tr><td>v</td></tr><tr><td>1</td></tr><tr><td>2</td></tr><tr><td>3</td></tr><tr><td>4</td></tr></table>	v	1	2	3	4
v																					
1																					
2																					
3																					
v																					
3																					
4																					
v																					
1																					
2																					
3																					
3																					
4																					
v																					
1																					
2																					
3																					
4																					

Regra prática: use sempre **UNION ALL**, a menos que você *precise* eliminar duplicatas. Aqui, como cada linha traz um rótulo diferente, não há duplicata possível — pagar pela deduplicação seria desperdício.

Nota

Esperamos: 3 filiais, 6 categorias, 4 segmentos de cliente (2 tipos × 2 gêneros) e 3 formas de pagamento. Se o número não bater, provavelmente há inconsistência de capitalização — algum **Credit card** escapou do INITCAP.

8.5 Fact_sales construída em passos

Passo 1 — a estrutura

A fato tem chave natural (**invoice_nk**), chaves estrangeiras para as cinco dimensões e as métricas:

```

1 DROP TABLE IF EXISTS dw.fact_sales CASCADE;
2 CREATE TABLE dw.fact_sales (
3     invoice_nk    VARCHAR(20) PRIMARY KEY,    -- dimensao degenerada
4     date_sk       INTEGER NOT NULL REFERENCES dw.dim_date(date_sk),
5     branch_sk     INTEGER NOT NULL REFERENCES dw.dim_branch(branch_sk),
6     product_sk    INTEGER NOT NULL REFERENCES dw.dim_product(product_sk),
7     customer_sk   INTEGER NOT NULL REFERENCES dw.dim_customer(customer_sk),
8     payment_sk    INTEGER NOT NULL REFERENCES dw.dim_payment(payment_sk),
9     unit_price    NUMERIC(10,2) NOT NULL,
10    quantity      INTEGER          NOT NULL,
11    total         NUMERIC(12,2) NOT NULL,
12    tax           NUMERIC(10,4) NOT NULL,
13    cogs          NUMERIC(12,2) NOT NULL,
14    gross_income  NUMERIC(10,4) NOT NULL,
15    rating        NUMERIC(4,1)  NOT NULL
16 );
17
18 CREATE INDEX ix_fs_date      ON dw.fact_sales(date_sk);
19 CREATE INDEX ix_fs_branch   ON dw.fact_sales(branch_sk);
20 CREATE INDEX ix_fs_product  ON dw.fact_sales(product_sk);
21 CREATE INDEX ix_fs_customer ON dw.fact_sales(customer_sk);
22 CREATE INDEX ix_fs_payment  ON dw.fact_sales(payment_sk);

```

Código 15: fact_sales — DDL

Nota

`invoice_nk` é uma **dimensão degenerada**: um identificador de negócio (o número do cupom) que não tem atributos próprios para justificar uma tabela dimensão inteira, então mora dentro da fato mesmo. É extremamente comum — números de pedido, de nota fiscal e de bilhete quase sempre acabam assim.

Passo 2 — chave e métricas

```

1 INSERT INTO dw.fact_sales (
2     invoice_nk,
3     unit_price, quantity, total, tax, cogs, gross_income, rating
4 )
5 SELECT
6     s.invoice_id,
7     s.unit_price, s.quantity, s.total,
8     s.tax_5pct,  s.cogs,      s.gross_income, s.rating
9 FROM staging.sales s;

```

Código 16: fact_sales — Passo 2 (só métricas)

Passo 3 — a chave de data**O que muda neste passo**

Adicionamos a chave de data (`date_sk`), derivada do `sale_ts`. Não é um JOIN — é um simples `TO_CHAR + CAST` que produz o inteiro `YYYYMMDD`.

```

1 INSERT INTO dw.fact_sales (
2     invoice_nk, date_sk,                                -- novo: date_sk
3     unit_price, quantity, total, tax, cogs, gross_income, rating
4 )
5 SELECT
6     s.invoice_id,
7     CAST(TO_CHAR(s.sale_ts, 'YYYYMMDD') AS INTEGER),    -- novo
8     s.unit_price, s.quantity, s.total,
9     s.tax_5pct, s.cogs, s.gross_income, s.rating
10 FROM staging.sales s;

```

Código 17: fact_sales — Passo 3 (soma date_sk)

Conceito-chave

Note que não fazemos JOIN com dim_date para achar o date_sk: como adotamos a convenção YYYYMMDD, basta formatar o timestamp. Em uma fato de bilhões de linhas, eliminar um JOIN inteiro da carga faz diferença enorme.

Passo 4 — as demais chaves via JOIN

O que muda neste passo

Entram as 4 surrogate keys restantes (branch_sk, product_sk, customer_sk, payment_sk), cada uma obtida por um JOIN com sua dimensão. Esse é o coração da carga de fato: trocar atributos naturais pelas surrogates.

```

1 TRUNCATE TABLE dw.fact_sales;                                -- novo
2
3 INSERT INTO dw.fact_sales (
4     invoice_nk, date_sk,
5     branch_sk, product_sk, customer_sk, payment_sk,        -- novo
6     unit_price, quantity, total, tax, cogs, gross_income, rating
7 )
8 SELECT
9     s.invoice_id,
10    CAST(TO_CHAR(s.sale_ts, 'YYYYMMDD') AS INTEGER),
11    db.branch_sk,                                           -- novo
12    dp.product_sk,                                           -- novo
13    dc.customer_sk,                                          -- novo
14    dpa.payment_sk,                                          -- novo
15    s.unit_price, s.quantity, s.total,
16    s.tax_5pct, s.cogs, s.gross_income, s.rating
17 FROM staging.sales s
18 JOIN dw.dim_branch db ON db.branch_code = s.branch        -- novo
19 JOIN dw.dim_product dp ON dp.product_line = s.product_line -- novo
20 JOIN dw.dim_customer dc ON dc.customer_type = s.customer_type -- novo
21                        AND dc.gender = s.gender           -- novo
22 JOIN dw.dim_payment dpa ON dpa.payment_type = s.payment;    -- novo
23
24 SELECT COUNT(*) FROM dw.fact_sales; -- 1000

```

Código 18: fact_sales — Passo 4 (completa)

No mercado

Esse padrão tem nome: **surrogate key lookup**. É a operação mais executada em qualquer carga de DW do mundo, e também a que mais quebra. Se uma venda chegar com uma filial

que não existe em `dim_branch`, o JOIN interno simplesmente *descarta* a linha — silenciosamente. Por isso a conferência final (`COUNT(*)` igual a 1.000) não é formalidade: é o teste que revela dado perdido. Em produção usa-se `LEFT JOIN` com uma linha “Desconhecido” de surrogate `-1` nas dimensões, para nunca perder um fato.

9 Automatizando com PL/pgSQL

Até aqui tudo foi SQL puro — declarativo, sem laços, sem variáveis. O PostgreSQL também aceita uma linguagem procedural embutida, o **PL/pgSQL**, que permite escrever funções, laços, condicionais e gatilhos *dentro* do banco. Vamos usá-la em duas situações típicas de um DW: uma **análise que precisa de memória entre linhas** (cursor) e uma **regra que precisa disparar sozinha** (trigger).

9.1 Um cursor para a curva ABC

Uma **análise ABC** (ou análise de Pareto) ordena os produtos do mais lucrativo para o menos lucrativo e vai acumulando o percentual do lucro total. Os produtos que somam os primeiros 80% formam a *curva A*; até 95%, a *curva B*; o resto, a *curva C*.

Repare no “vai acumulando”: para classificar uma linha é preciso saber o que aconteceu nas linhas anteriores. É exatamente aí que um **cursor** se encaixa — ele permite percorrer o resultado de uma consulta *uma linha por vez*, guardando estado em variáveis.

```
1 DROP TABLE IF EXISTS dw.abc_product;  
2 CREATE TABLE dw.abc_product (  
3     product_line  VARCHAR(40) PRIMARY KEY,  
4     lucro         NUMERIC(12,2),  
5     perc_acumulado NUMERIC(6,2),  
6     curva        CHAR(1)  
7 );
```

Código 19: Tabela de destino da análise ABC

```

1 CREATE OR REPLACE FUNCTION dw.calcula_curva_abc()
2 RETURNS INTEGER AS $$
3 DECLARE
4     -- 1) declaracao do cursor: a consulta que sera percorrida
5     c_prod CURSOR FOR
6         SELECT p.product_line,
7             SUM(f.gross_income)::NUMERIC(12,2) AS lucro
8         FROM dw.fact_sales f
9         JOIN dw.dim_product p ON p.product_sk = f.product_sk
10        GROUP BY p.product_line
11        ORDER BY lucro DESC;
12
13     r_prod    RECORD;          -- variavel que recebe cada linha
14     v_total   NUMERIC(12,2);    -- lucro total de todas as categorias
15     v_acum    NUMERIC(12,2) := 0;
16     v_perc    NUMERIC(6,2);
17     v_curva   CHAR(1);
18     v_linhas  INTEGER := 0;
19 BEGIN
20     TRUNCATE TABLE dw.abc_product;
21
22     SELECT SUM(gross_income) INTO v_total FROM dw.fact_sales;
23
24     OPEN c_prod;                -- 2) abre o cursor
25     LOOP
26         FETCH c_prod INTO r_prod;    -- 3) pega a proxima linha
27         EXIT WHEN NOT FOUND;        -- 4) acabou? sai do laço
28
29         v_acum := v_acum + r_prod.lucro;
30         v_perc := ROUND(100 * v_acum / v_total, 2);
31
32         IF v_perc <= 80 THEN v_curva := 'A';
33         ELSIF v_perc <= 95 THEN v_curva := 'B';
34         ELSE v_curva := 'C';
35         END IF;
36
37         INSERT INTO dw.abc_product
38         VALUES (r_prod.product_line, r_prod.lucro, v_perc, v_curva);
39
40         v_linhas := v_linhas + 1;
41         RAISE NOTICE 'Categoria % -> curva % (acumulado %%)',
42             r_prod.product_line, v_curva, v_perc;
43     END LOOP;
44     CLOSE c_prod;                -- 5) fecha o cursor
45
46     RETURN v_linhas;
47 END;
48 $$ LANGUAGE plpgsql;
49
50 -- executa e confere
51 SELECT dw.calcula_curva_abc();
52 SELECT * FROM dw.abc_product ORDER BY perc_acumulado;

```

Código 20: Procedimento com cursor em PL/pgSQL

Conceito-chave

O ciclo de vida de um cursor tem sempre os mesmos cinco momentos:

1. DECLARE — associa um nome a uma consulta.
2. OPEN — executa a consulta e posiciona o ponteiro antes da primeira linha.

3. `FETCH ... INTO` — avança uma linha e copia os valores para variáveis.
4. `EXIT WHEN NOT FOUND` — `FOUND` é uma variável booleana que o PL/pgSQL atualiza automaticamente; quando o `FETCH` não encontra mais nada, ela vira falsa.
5. `CLOSE` — libera os recursos.

O `RAISE NOTICE` imprime mensagens no painel de mensagens do pgAdmin — é a forma mais simples de depurar PL/pgSQL. O `%` é substituído pelos argumentos na ordem; para imprimir um `%` literal, escreve-se `%%`.

No mercado

Cursors têm má fama — e com razão: processar linha a linha é ordens de magnitude mais lento que uma operação em conjunto. Um DBA experiente resolveria essa mesma curva ABC com uma *window function* (`SUM(...) OVER (ORDER BY lucro DESC)`), sem nenhum laço. Use cursor quando: (a) cada linha depende do resultado do processamento da anterior de um jeito que o SQL não expressa, (b) você precisa chamar um serviço externo por registro, ou (c) precisa processar milhões de linhas em lotes, com `COMMIT` intermediário, para não estourar memória. Fora disso, prefira SQL em conjunto.

9.2 Um trigger para auditar a fato

Um **trigger** (gatilho) é um procedimento que o banco dispara *automaticamente* quando algo acontece em uma tabela. Ninguém o chama: ele reage. É o mecanismo natural para regras que precisam valer *sempre*, independentemente de quem escreveu o `INSERT`.

Vamos criar um que faz duas coisas ao inserir na fato: valida a linha e registra a operação em uma tabela de log.

```

1 DROP TABLE IF EXISTS dw.fact_sales_log;
2 CREATE TABLE dw.fact_sales_log (
3     log_id      SERIAL PRIMARY KEY,
4     invoice_nk  VARCHAR(20),
5     operacao    VARCHAR(10),
6     usuario     VARCHAR(60),
7     momento     TIMESTAMP
8 );
9
10 CREATE OR REPLACE FUNCTION dw.audita_fact_sales()
11 RETURNS TRIGGER AS $$
12 BEGIN
13     -- validacao: nao aceitamos venda sem itens
14     IF NEW.quantity <= 0 THEN
15         RAISE EXCEPTION 'Venda % rejeitada: quantidade invalida (%)',
16             NEW.invoice_nk, NEW.quantity;
17     END IF;
18
19     -- validacao: a nota do cliente vai de 4 a 10 neste dataset
20     IF NEW.rating < 4 OR NEW.rating > 10 THEN
21         RAISE EXCEPTION 'Venda % rejeitada: rating fora da faixa (%)',
22             NEW.invoice_nk, NEW.rating;
23     END IF;
24
25     -- auditoria
26     INSERT INTO dw.fact_sales_log (invoice_nk, operacao, usuario, momento)
27     VALUES (NEW.invoice_nk, TG_OP, CURRENT_USER, NOW());
28
29     RETURN NEW; -- devolve a linha para que o INSERT prossiga
30 END;
31 $$ LANGUAGE plpgsql;

```

Código 21: Tabela de log e função do trigger

```

1 DROP TRIGGER IF EXISTS trg_audita_fact_sales ON dw.fact_sales;
2
3 CREATE TRIGGER trg_audita_fact_sales
4     BEFORE INSERT ON dw.fact_sales
5     FOR EACH ROW
6     EXECUTE FUNCTION dw.audita_fact_sales();
7
8 -- teste 1: linha valida -> entra e gera log
9 INSERT INTO dw.fact_sales VALUES
10     ('999-99-9999', 20190315, 1, 1, 1, 1, 10.00, 2, 21.00, 1.00, 20.00, 1.00, 9.0);
11
12 -- teste 2: linha invalida -> o trigger aborta o INSERT
13 INSERT INTO dw.fact_sales VALUES
14     ('888-88-8888', 20190315, 1, 1, 1, 1, 10.00, 0, 21.00, 1.00, 20.00, 1.00, 9.0);
15
16 SELECT * FROM dw.fact_sales_log;
17
18 -- limpeza do teste
19 DELETE FROM dw.fact_sales WHERE invoice_nk = '999-99-9999';

```

Código 22: Criando e testando o trigger

Conceito-chave

Anatomia de um trigger no PostgreSQL:

- A **função** retorna o tipo especial TRIGGER e recebe, de graça, as variáveis NEW (a linha

depois da operação) e OLD (antes dela), além de TG_OP, que contém 'INSERT', 'UPDATE' ou 'DELETE'.

- O **trigger** amarra essa função a uma tabela e a um momento: BEFORE (pode alterar ou rejeitar a linha) ou AFTER (a linha já foi gravada; serve para efeitos colaterais).
- FOR EACH ROW dispara uma vez por linha; FOR EACH STATEMENT dispara uma vez por comando, mesmo que ele afete mil linhas.
- Em um trigger BEFORE, retornar NEW deixa a operação seguir; retornar NULL a cancela silenciosamente; RAISE EXCEPTION a cancela com erro e desfaz a transação inteira.

No mercado

Em DWs, triggers aparecem sobretudo em três papéis: **auditoria** (quem carregou o quê e quando — exigência de auditores e de normas como SOX e LGPD), **qualidade** (barrar dado absurdo antes que ele contamine um dashboard) e **manutenção de agregados** (atualizar uma tabela de totais a cada inserção). O contraponto: um trigger FOR EACH ROW numa carga de 50 milhões de linhas multiplica o tempo do processo. Por isso muitas equipes desabilitam triggers durante a carga em massa (ALTER TABLE ... DISABLE TRIGGER) e rodam as validações em lote depois.

10 Consultas Analíticas

Chegamos ao propósito final do DW: responder perguntas de negócio com SQL simples. Todas as consultas abaixo seguem a mesma receita — **fato JOIN dimensões**, GROUP BY em atributos de dimensão, agregações sobre as métricas da fato. Essa uniformidade é o grande prêmio da modelagem dimensional.

10.1 P1 — Receita por filial e mês

```

1 SELECT
2     d.year,
3     d.month_name,
4     b.branch_code          AS filial,
5     b.city,
6     COUNT(*)               AS vendas,
7     SUM(f.total)::NUMERIC(12,2) AS receita,
8     ROUND(AVG(f.total), 2)   AS ticket_medio
9 FROM dw.fact_sales f
10 JOIN dw.dim_date d ON d.date_sk = f.date_sk
11 JOIN dw.dim_branch b ON b.branch_sk = f.branch_sk
12 GROUP BY d.year, d.month, d.month_name, b.branch_code, b.city
13 ORDER BY d.month, filial;
```

Código 23: Receita mensal por filial

O que está acontecendo. Esta é a consulta mais clássica de um DW: um cruzamento *tempo* × *local*. Partimos da fato e trazemos dois contextos — a data (*dim_date*) e a filial (*dim_branch*). O GROUP BY define o nível em que o detalhe será colapsado: cada linha do resultado passa a representar “um mês de uma filial”, não mais uma venda.

As três agregações respondem a perguntas diferentes sobre o mesmo grupo:

- COUNT(*) — *volume*: quantas vendas aconteceram.
- SUM(f.total) — *faturamento*: quanto dinheiro entrou.

- `AVG(f.total)` — *ticket médio*: quanto vale, em média, cada compra.

Volume e faturamento contam histórias diferentes: uma filial pode faturar mais vendendo *menos vezes*, se cada compra for maior. É por isso que o ticket médio quase nunca falta em um relatório de varejo.

Repare em `GROUP BY d.year, d.month, d.month_name`: agrupamos por `d.month` (o número) mesmo sem exibi-lo, porque é ele que permite ordenar os meses cronologicamente. Ordenar por `month_name` colocaria Abril antes de Janeiro — ordem alfabética.

10.2 P2 — Categorias mais lucrativas

```

1 SELECT
2     p.product_line                AS categoria,
3     COUNT(*)                     AS vendas,
4     SUM(f.quantity)              AS itens_vendidos,
5     SUM(f.total)::NUMERIC(12,2)  AS receita,
6     SUM(f.gross_income)::NUMERIC(12,2) AS lucro_bruto,
7     ROUND(AVG(f.rating), 2)      AS avaliacao_media
8 FROM dw.fact_sales f
9 JOIN dw.dim_product p ON p.product_sk = f.product_sk
10 GROUP BY p.product_line
11 ORDER BY lucro_bruto DESC;
```

Código 24: Ranking de categorias por lucro bruto

O que está acontecendo. Aqui um único JOIN basta, porque a pergunta tem uma só dimensão: produto. O resultado terá exatamente 6 linhas — uma por categoria do dataset.

O ponto interessante é a coexistência de `receita` e `lucro_bruto`. Elas não andam necessariamente juntas: uma categoria de alto giro e margem apertada pode liderar a receita e ficar no meio do ranking de lucro. Ordenamos por lucro porque, no fim, é o lucro que paga a operação — mas manter as duas colunas lado a lado é o que torna a comparação visível.

`SUM(f.quantity)` soma *itens*, enquanto `COUNT(*)` conta *cupons*. Dividir um pelo outro daria os itens por compra — outro indicador clássico. E o `AVG(f.rating)` coloca a satisfação do cliente ao lado do dinheiro: uma categoria muito lucrativa e mal avaliada é um risco de médio prazo, não uma vitória.

10.3 P3 — Comportamento por gênero e tipo de cliente

```

1 SELECT
2     c.customer_type              AS tipo,
3     c.gender                    AS genero,
4     COUNT(*)                    AS vendas,
5     ROUND(AVG(f.total), 2)      AS ticket_medio,
6     ROUND(AVG(f.rating), 2)    AS avaliacao_media,
7     ROUND(AVG(f.quantity), 2)  AS itens_medio
8 FROM dw.fact_sales f
9 JOIN dw.dim_customer c ON c.customer_sk = f.customer_sk
10 GROUP BY c.customer_type, c.gender
11 ORDER BY tipo, genero;
```

Código 25: Análise por segmento de cliente

O que está acontecendo. O `GROUP BY` com *duas* colunas da mesma dimensão cria uma grade: Member/Female, Member/Male, Normal/Female, Normal/Male — 4 linhas, todas as combinações que existem nos dados.

Note que aqui quase tudo é média, não soma. É deliberado: os quatro segmentos não têm o mesmo tamanho, e comparar somas seria comparar o tamanho dos grupos, não o comportamento deles. A média neutraliza isso. Mantivemos o `COUNT(*)` justamente para saber *sobre quantas*

vendas cada média foi calculada — uma média de 5 vendas não merece a mesma confiança que uma de 300.

A pergunta de negócio por trás: o programa de fidelidade (*Member*) está entregando o que promete? Se o ticket médio dos membros não for maior que o dos clientes normais, o desconto que eles recebem está saindo de graça.

10.4 P4 — Dia da semana mais forte

```

1 SELECT
2     d.day_of_week           AS dia_semana,
3     d.is_weekend           AS fim_de_semana,
4     COUNT(*)               AS vendas,
5     SUM(f.total)::NUMERIC(12,2) AS receita,
6     ROUND(AVG(f.rating), 2) AS avaliacao_media
7 FROM dw.fact_sales f
8 JOIN dw.dim_date d ON d.date_sk = f.date_sk
9 GROUP BY d.day_of_week, d.is_weekend
10 ORDER BY receita DESC;
```

Código 26: Vendas por dia da semana

O que está acontecendo. Esta consulta mostra por que vale a pena ter uma `dim_date` rica. Os atributos `day_of_week` e `is_weekend` não existem em lugar nenhum do CSV original — nós os calculamos uma única vez, ao gerar a dimensão. Agora qualquer analista responde “sábado vende mais?” sem saber o que é `EXTRACT(DOW ...)`.

É o oposto de agregar no tempo: aqui *ignoramos* a linha do tempo e empilhamos todas as terças-feiras de 2019 numa única linha, procurando um padrão **cíclico**. Esse tipo de resposta alimenta decisões muito concretas: escala de funcionários, dia da reposição de estoque, quando lançar uma promoção.

`is_weekend` entra no `GROUP BY` sem alterar os grupos — ele é totalmente determinado pelo dia da semana. Está ali só para permitir sua exibição, já que toda coluna do `SELECT` que não está dentro de uma função de agregação precisa estar no `GROUP BY`. É um truque comum e inofensivo.

Nota

Ordenamos por receita, e não pela ordem natural dos dias, porque a pergunta é “qual o dia mais forte?”. Um relatório para acompanhamento diário faria o contrário: ordenaria de segunda a domingo, para que a forma da semana ficasse visível.

11 Cubo OLAP: a última peça

Todas as consultas da seção anterior tinham a mesma forma. Trocamos a dimensão do `GROUP BY` — filial, produto, cliente, dia da semana — e a estrutura continuou idêntica. Isso não é coincidência: é o sintoma de que existe uma *única* estrutura por trás de todas elas. Essa estrutura é o **cubo**.

Um **cubo OLAP** (ou *cubo de dados*) é a visão do star schema como um espaço de várias dimensões, onde cada *célula* guarda uma métrica já agregada. Se as dimensões são *tempo*, *produto* e *filial*, então a célula (Março, Bebidas, Filial B) contém a receita daquela combinação — pronta, sem precisar somar nada na hora.

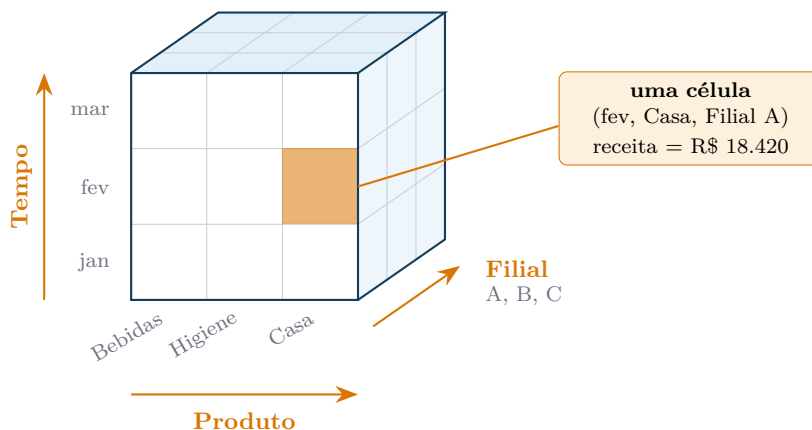


Figura 12: O cubo de dados. Cada eixo é uma dimensão; cada célula, uma métrica já agregada. Com 5 dimensões o desenho seria impossível, mas a ideia é a mesma — por isso se fala em *hipercubo*.

11.1 As quatro operações do cubo

O vocabulário abaixo é usado em qualquer ferramenta de BI do mercado. Você já executou todas elas em SQL, sem saber os nomes:

- **Slice** (fatiar) — fixa *uma* dimensão em um valor e olha o plano resultante. Ex.: “só março”. Em SQL: `WHERE d.month = 3`.
- **Dice** (recortar) — restringe *várias* dimensões a subconjuntos, formando um cubo menor. Ex.: “filiais A e B, no primeiro trimestre, categorias de bebida”. Em SQL: um `WHERE` com vários `IN`.
- **Drill-down / Roll-up** — desce ou sobe um nível de hierarquia. Em SQL: trocar o que está no `GROUP BY` (de `year` para `month`, ou de `month` para `day`).
- **Pivot** (girar) — troca quais dimensões ficam nas linhas e quais ficam nas colunas. É o que a tabela dinâmica do Excel faz quando você arrasta um campo.



Figura 13: Hierarquia da dimensão tempo. *Drill-down* desce até o detalhe; *roll-up* sobe agregando. Toda dimensão bem construída tem pelo menos uma hierarquia dessas.

11.2 Cubo no próprio PostgreSQL: GROUPING SETS, ROLLUP e CUBE

Não é preciso um servidor OLAP dedicado para experimentar a ideia. O PostgreSQL implementa os operadores do padrão SQL que calculam *vários níveis de agregação em uma única passada*:


```

1 -- ROLLUP: hierarquia. Gera (filial, mes), (filial), e o total geral.
2 SELECT
3     b.branch_code           AS filial,
4     d.month_name           AS mes,
5     SUM(f.total)::NUMERIC(12,2) AS receita
6 FROM dw.fact_sales f
7 JOIN dw.dim_branch b ON b.branch_sk = f.branch_sk
8 JOIN dw.dim_date d ON d.date_sk = f.date_sk
9 GROUP BY ROLLUP (b.branch_code, d.month_name)
10 ORDER BY b.branch_code, d.month_name;
11
12 -- CUBE: TODAS as combinacoes possiveis das dimensoes listadas.
13 -- (filial, pagamento), (filial), (pagamento) e o total geral.
14 SELECT
15     COALESCE(b.branch_code, 'TODAS') AS filial,
16     COALESCE(p.payment_type, 'TODOS') AS pagamento,
17     SUM(f.total)::NUMERIC(12,2) AS receita
18 FROM dw.fact_sales f
19 JOIN dw.dim_branch b ON b.branch_sk = f.branch_sk
20 JOIN dw.dim_payment p ON p.payment_sk = f.payment_sk
21 GROUP BY CUBE (b.branch_code, p.payment_type)
22 ORDER BY filial, pagamento;

```

Código 27: Subtotais automáticos com ROLLUP e CUBE

Conceito-chave

Com ROLLUP (a, b) o banco calcula os grupos (a,b), (a) e () — ou seja, respeita uma *hierarquia*, ideal para subtotais de relatório. Com CUBE (a, b) ele calcula *todas* as 2ⁿ combinações, incluindo (b) sozinho. Nas linhas de subtotal, as colunas agregadas vêm como NULL — daí o COALESCE para exibir “TODAS”. É literalmente um cubo materializado sob demanda, em um único SELECT.

11.3 Ferramentas do mercado

Cubos podem ser *pré-calculados e armazenados* (MOLAP), *calculados na hora sobre o star schema* (ROLAP) ou uma mistura dos dois (HOLAP). Cada família tem suas ferramentas:

Categoria	Ferramentas	Observação
Servidores OLAP clássicos	Microsoft Analysis Services (SSAS), Oracle Essbase, IBM Cognos TM1, SAP BW	Cubos materializados, linguagem MDX. Ainda dominantes em grandes corporações.
OLAP open source	Mondrian / Pentaho, Apache Kylin, Apache Druid, ClickHouse, Apache Doris	Kylin pré-computa cubos sobre Hadoop; Druid e ClickHouse resolvem agregações em tempo real.
Camada semântica moderna	Cube (cube.dev), dbt Semantic Layer, AtScale, Looker (LookML)	Definem métricas e hierarquias uma vez e geram o SQL para quem consultar.
Ferramentas de visualização	Power BI, Tableau, Qlik Sense, Metabase, Apache Superset, Looker Studio	Trazem o cubo para o usuário final: arrastar campos é fazer <i>pivot</i> e <i>drill-down</i> .
Planilha	Tabela dinâmica do Excel / Google Sheets	A interface OLAP mais usada do planeta — e a mais subestimada.
Dentro do próprio banco	GROUPING SETS, ROLLUP, CUBE, <i>materialized views</i> , extensão cube	É o que fizemos aqui: PostgreSQL fazendo papel de motor OLAP.

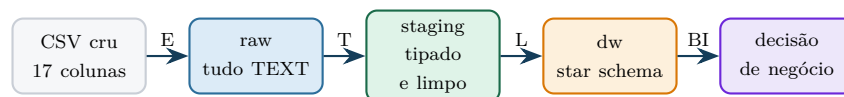
No mercado

Há uma tendência clara: o cubo materializado à moda antiga perdeu espaço. Bancos colunares modernos varrem bilhões de linhas rápido o suficiente para dispensar o pré-cálculo, e a complexidade de manter cubos atualizados deixou de compensar. O que sobreviveu foi o *vocabulário* — slice, dice, drill-down, roll-up — e a ideia de uma camada semântica onde “receita líquida” seja definida uma única vez para a empresa inteira. Se cada área define a métrica do seu jeito, o dashboard vira briga de reunião, não decisão.

12 Conclusão

Começamos com um arquivo de texto de 1.000 linhas — o tipo de coisa que qualquer pessoa abriria no Excel e fecharia dez minutos depois sem ter descoberto nada. Terminamos com um Data Warehouse funcional, capaz de responder perguntas de negócio em uma consulta de dez linhas.

O caminho percorrido foi este:



E, no caminho, ficaram algumas ideias que valem mais do que o código:

- **Camadas existem para que o erro seja barato.** A **raw** aceita lixo justamente para que a falha aconteça no seu **SELECT**, e não na importação — com o dado já dentro de casa, disponível para investigação.
- **Desnormalizar é uma escolha, não um descuido.** Você aprendeu as três formas normais para poder abandoná-las com consciência quando o objetivo deixa de ser integridade e passa a ser velocidade de leitura.
- **O grão é irreversível.** É a única decisão do projeto que você não conserta depois: o detalhe que não foi guardado não volta.
- **SQL bem modelado dispensa ferramenta cara.** Fizemos ETL, dimensões, fato, auditoria e cubo com PostgreSQL e pgAdmin — sem uma linha de Python.
- **O DW não termina em uma tabela.** Ele termina em alguém decidindo em qual dia contratar mais gente para o caixa.

12.1 Para onde ir a partir daqui

- **Mais dados, mais dor:** refaça o exercício com Olist, Northwind ou TPC-H. Com múltiplas tabelas de origem aparecem os problemas de verdade — chaves que não batem, duplicatas, dados chegando fora de ordem.
- **SCD Tipo 2:** com um ID real de cliente, implemente histórico de mudanças. É o conceito que separa quem “já mexeu com DW” de quem sabe modelar.
- **Cargas incrementais:** pare de dar TRUNCATE e carregue só o que mudou desde ontem. Muda tudo na arquitetura.
- **Orquestração:** transforme os blocos SQL em modelos do dbt ou tarefas do Airflow, com dependências, testes e reexecução automática.
- **Visualização:** conecte Metabase, Superset ou Power BI ao schema `dw` e veja que, com um star schema bem feito, o dashboard sai quase sozinho.
- **Otimização:** particionamento por data, índices BRIN e *materialized views* — os recursos que fazem a diferença quando a fato passa de mil para cem milhões de linhas.

*A modelagem dimensional é intemporal.
As ferramentas mudam; os princípios permanecem.*